# CS 301
# High-Performance Computing

## Lab 01 - CPU architecture, Memory Hierarchy, and Performance Measurement

Khushi Pandya (202301406)
Aalok Thakkar (202301429)

February 3, 2026

# Contents


## 1 Introduction

## 2 Hardware Details

### 2.1 Hardware Details for LAB207 PCs

### 2.2 Hardware Details for HPC Cluster (Node gics1)

## 3 Problem Description

## 4 Graphical Results

### 4.1 Vector Copy Operation : $a[i] = b[i]$

#### 4.1.1 Throughput and Bandwidth using Lab PC

#### 4.1.2 Throughput and FLOPs using HPC Cluster

### 4.2 Vector Scaling Operation : $a[i] = k \times b[i]$

#### 4.2.1 Throughput and Bandwidth using Lab PC

#### 4.2.2 Throughput and FLOPs using HPC Cluster

### 4.3 Vector Sum Operation : $a[i] = b[i] + c[i]$

#### 4.3.1 Throughput and Bandwidth using Lab PC

#### 4.3.2 Throughput and FLOPs using HPC Cluster

### 4.4 Vector Triad Operation : $a[i] = b[i] + c[i] \times d[i]$

#### 4.4.1 Throughput and Bandwidth using Lab PC

#### 4.4.2 Throughput and FLOPs using HPC Cluster

## 5 Analytical Tasks

## 6 Conclusion

# 7 Introduction

In this experiment, we study how CPU architecture and memory hierarchy affect the performance of a program. To understand this, simple vector operations such as copy, scale, add, and triad are implemented. These operations mainly involve accessing large arrays stored in memory. By measuring execution time, throughput, and memory bandwidth for different problem sizes, we can observe how cache memory and main memory influence overall performance.

# 8 Hardware Details

## 8.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x
- L1d cache: 288K
- L1i cache: 192K
- L2 cache: 7.5M

- L3 cache: 18M
- NUMA node0 CPU(s): 0-11
- Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

### 8.2 Hardware Details for HPC Cluster (Node gics1)

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 16
- On-line CPU(s) list: 0-15
- Thread(s) per core: 1
- Core(s) per socket: 8
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 1288.726
- BogoMIPS: 5204.73
- Virtualization: VT-x
- L1d cache: 32K
- L1i cache: 32K
- L2 cache: 256K

- L3 cache: 20480K
- NUMA node0 CPU(s): 0-7
- NUMA node1 CPU(s): 8-15

# 9 Problem Description

This lab uses a STREAM-like benchmark to evaluate memory performance of the CPU. The benchmark focuses on measuring memory bandwidth using simple vector operations. These operations are designed in such a way that performance is mainly limited by memory access rather than computation.

We consider a simple particle-based computation with the following setup:

- A two-dimensional square domain of size 1 cm $\times$ 1 cm.
- A total of $N_p$ particles randomly distributed within the domain.
- A conceptual mesh of size $N_x \times N_y$ defining the spatial extent.
- No mesh-based computation is performed in this experiment.

Each particle is characterized by its position $(x, y)$ and velocity $v$. All particles have the same constant mass $m$. The kinetic energy of each particle is given by:

$$E = \frac{1}{2}mv^2$$

The main purpose of this setup is to generate a memory-intensive workload that helps analyze performance for different working data sizes.

## Data Structures

The implementation uses arrays of length $N_p$ to store particle information:

- $x[p]$, $y[p]$: particle position coordinates
- $v[p]$: particle velocity
- $E[p]$: kinetic energy of each particle
- Particle mass $m$ is constant for all particles and is set to $m = 2$

## Initialization

Each particle is initialized using random values:

- Position coordinates $x[p]$ and $y[p]$ are generated in the range $[0, 1]$
- Velocity $v[p]$ is generated using a random function

This ensures that particles are uniformly distributed within the domain and have different velocities.

## Benchmark Kernels

To analyze memory performance, STREAM-like benchmark kernels are implemented. These kernels perform simple operations on large arrays and stress the memory subsystem.

The following benchmark kernels are implemented:

- **Copy**: assigns values from one array to another
- **Scale**: multiplies each element of an array by a constant
- **Add**: adds corresponding elements of two arrays
- **Triad**: performs multiplication and addition in a single operation

These kernels are useful for studying memory bandwidth because they involve multiple memory reads and writes.

## Benchmarking Methodology

The benchmark is executed for problem sizes ranging from small to very large values. For each problem size:

- The kernel is executed multiple times
- The total number of operations is kept approximately constant

This allows fair comparison of performance across different data sizes.

Execution time is measured using high-resolution timing functions. Using the measured time, throughput and memory bandwidth are calculated. These values are then plotted against problem size to observe performance trends.

# 10 Graphical Results

## 10.1 Vector Copy Operation : $a[i] = b[i]$

### 10.1.1 Throughput and Bandwidth using Lab PC

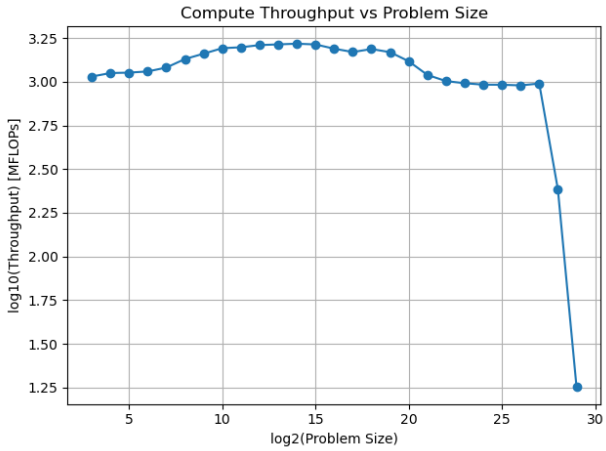


Figure 1: Throughput vs. Problem Size for the vector copy operation using Lab PC.

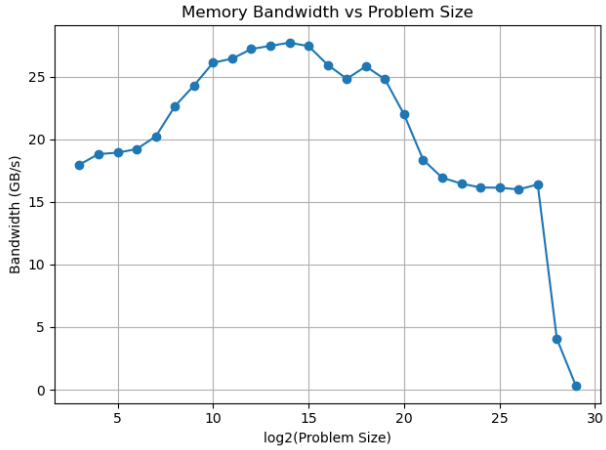


Figure 2: Memory Bandwidth vs. Problem Size for the vector copy operation using Lab PC.

### 10.1.2 Throughput and FLOPs using HPC Cluster

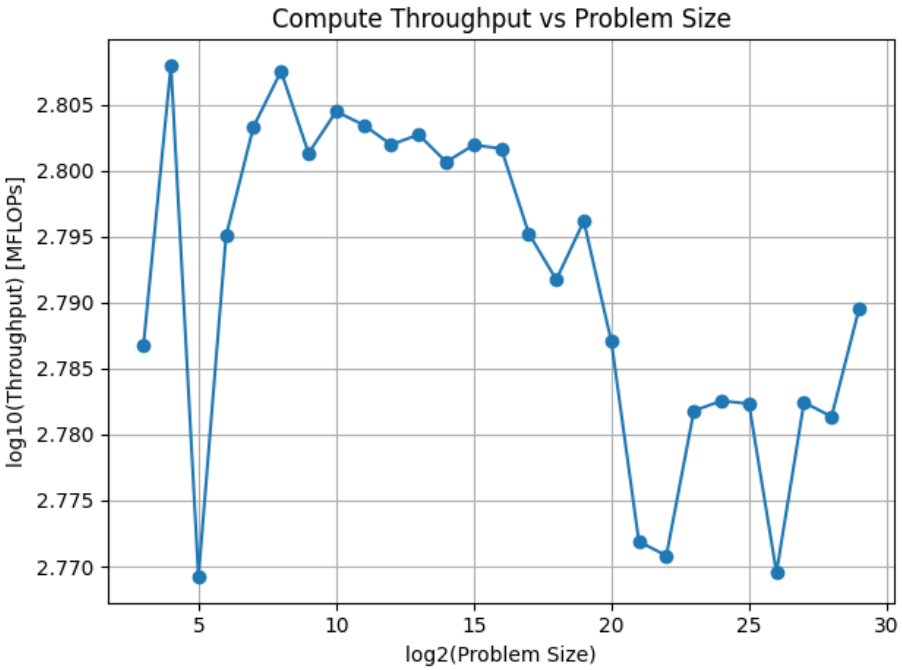


Figure 3: Throughput vs. Problem Size for the vector copy operation using HPC Cluster.

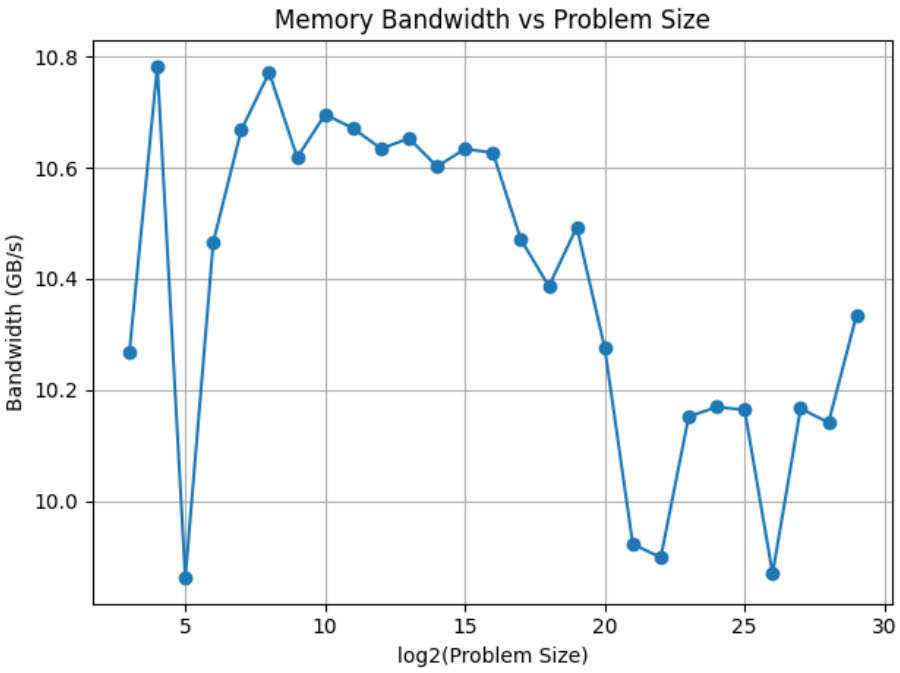


Figure 4: FLOPs vs. Problem Size for the vector copy operation using HPC Cluster.

## 10.2 Vector Scaling Operation : $a[i] = k * b[i]$

### 10.2.1 Throughput and Bandwidth using Lab PC

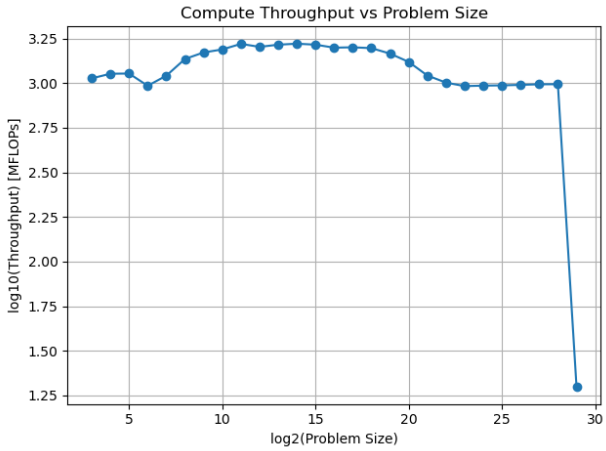


Figure 5: Throughput vs. Problem Size for the vector scaling operation using Lab PC.

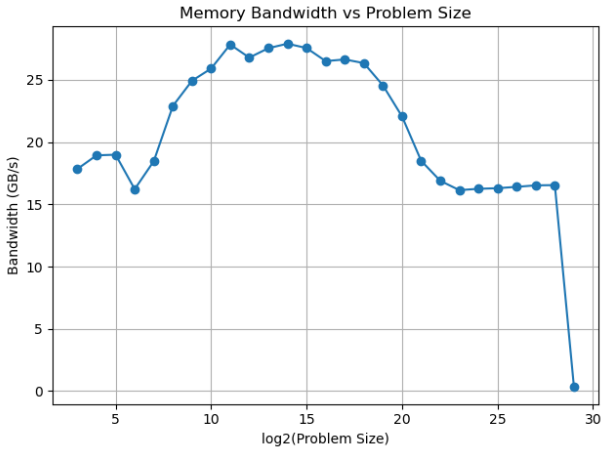


Figure 6: Memory Bandwidth vs. Problem Size for the vector scaling operation using Lab PC.

### 10.2.2 Throughput and FLOPs using HPC Cluster

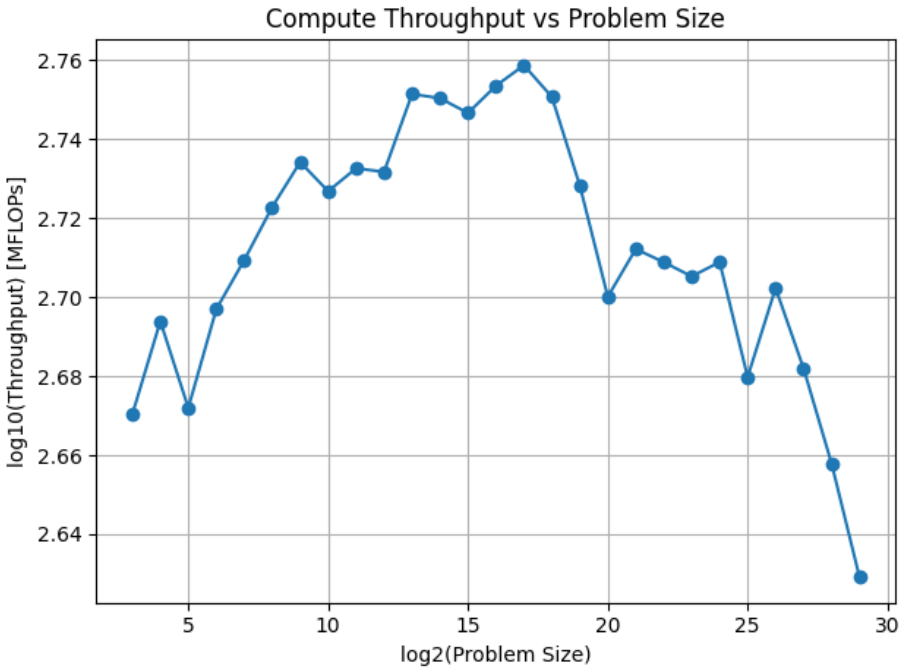


Figure 7: Throughput vs. Problem Size for the vector triad operation using HPC Cluster

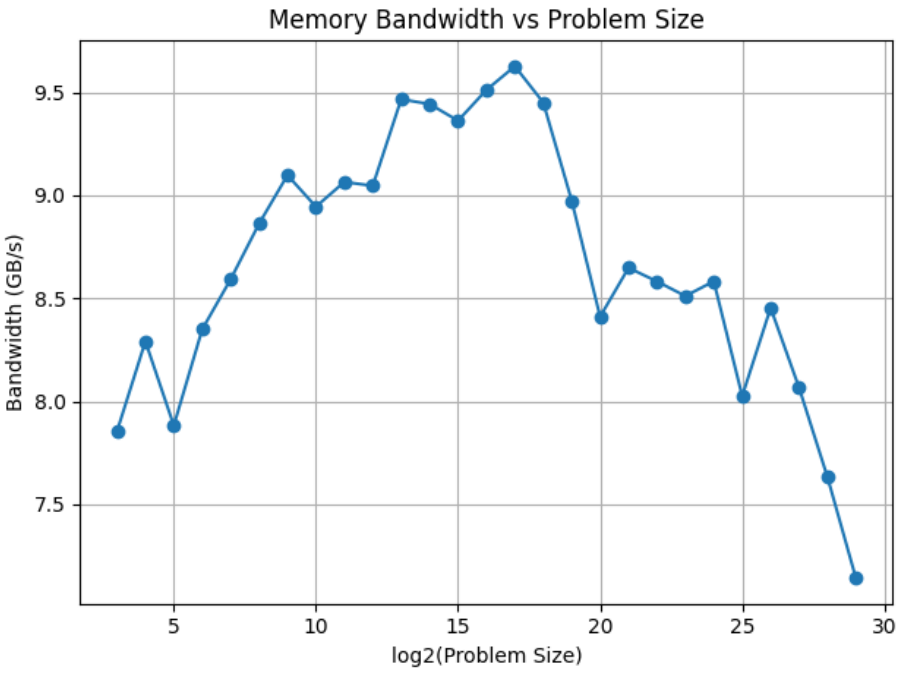


Figure 8: FLOPs vs. Problem Size for the vector triad operation using HPC Cluster.

## 10.3 Vector Sum Operation : $a[i] = b[i] + c[i]$

### 10.3.1 Throughput and Bandwidth using Lab PC

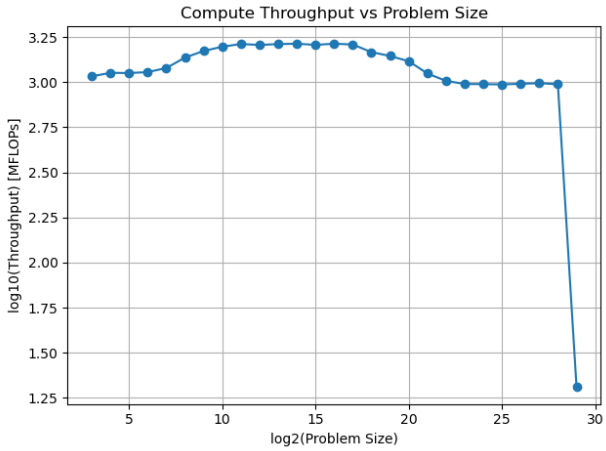


Figure 9: Throughput vs. Problem Size for the vector sum operation using Lab PC.

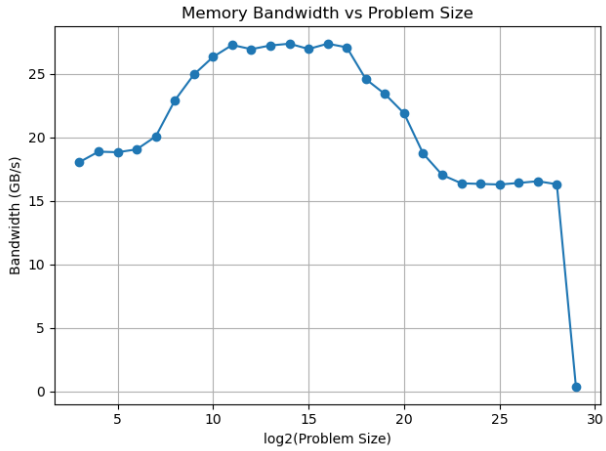


Figure 10: Memory Bandwidth vs. Problem Size for the vector sum operation using Lab PC.

### 10.3.2 Throughput and FLOPs using HPC Cluster

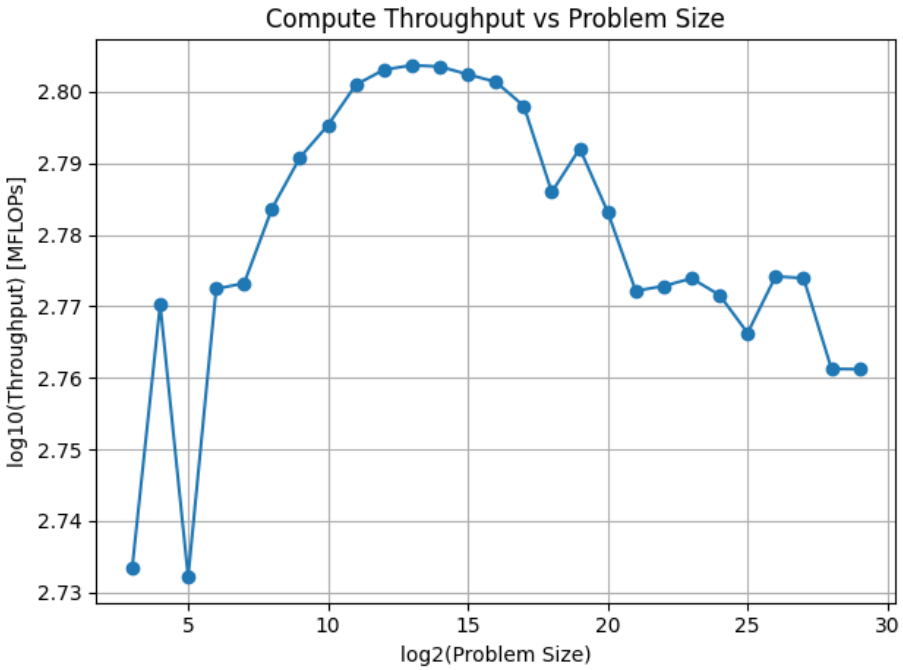


Figure 11: Throughput vs. Problem Size for the vector sum operation using HPC Cluster

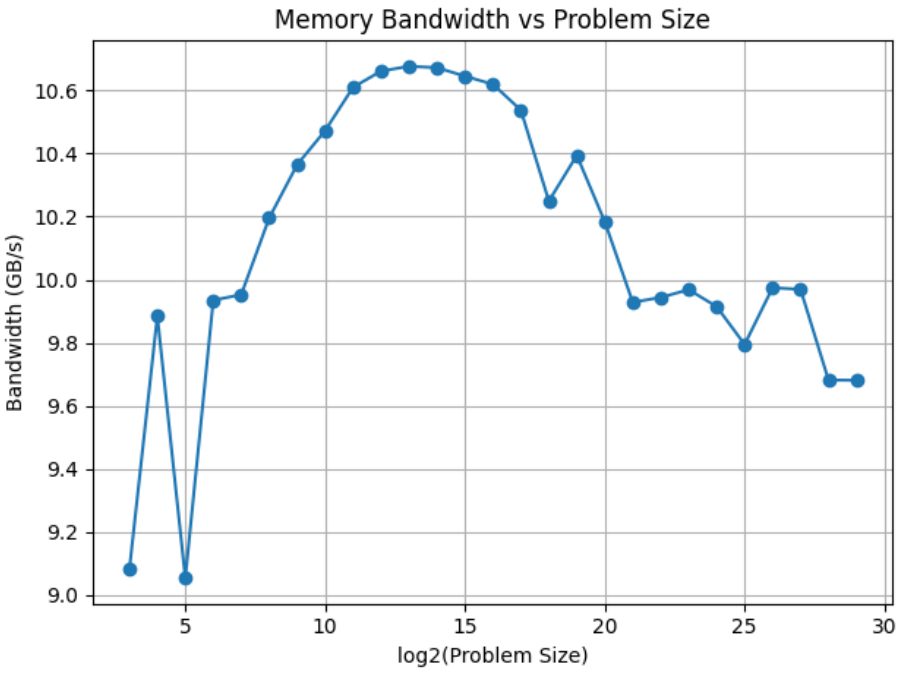


Figure 12: FLOPs vs. Problem Size for the vector sum operation using HPC Cluster.

## 10.4 Vector Triad Operation : $a[i] = b[i] + c[i] * d[i]$

### 10.4.1 Throughput and Bandwidth using Lab PC

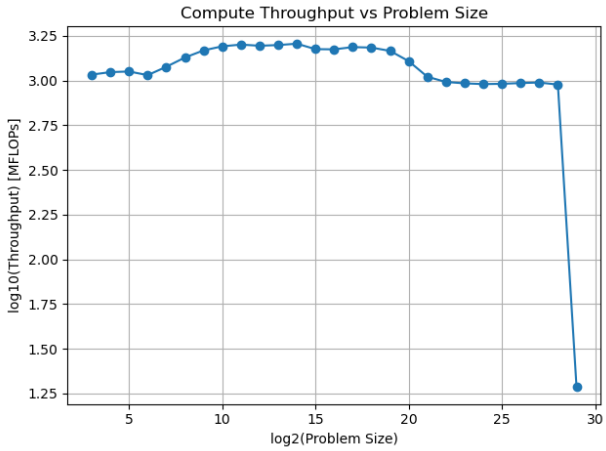


Figure 13: Throughput vs. Problem Size for the vector triad operation using Lab PC.

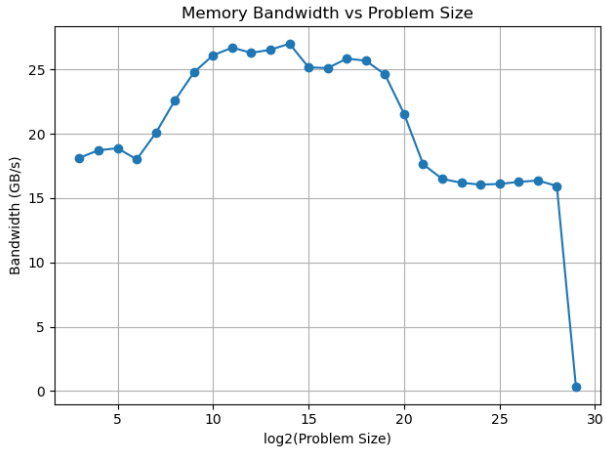


Figure 14: Memory Bandwidth vs. Problem Size for the vector triad operation using Lab PC.

### 10.4.2 Throughput and FLOPs using HPC Cluster

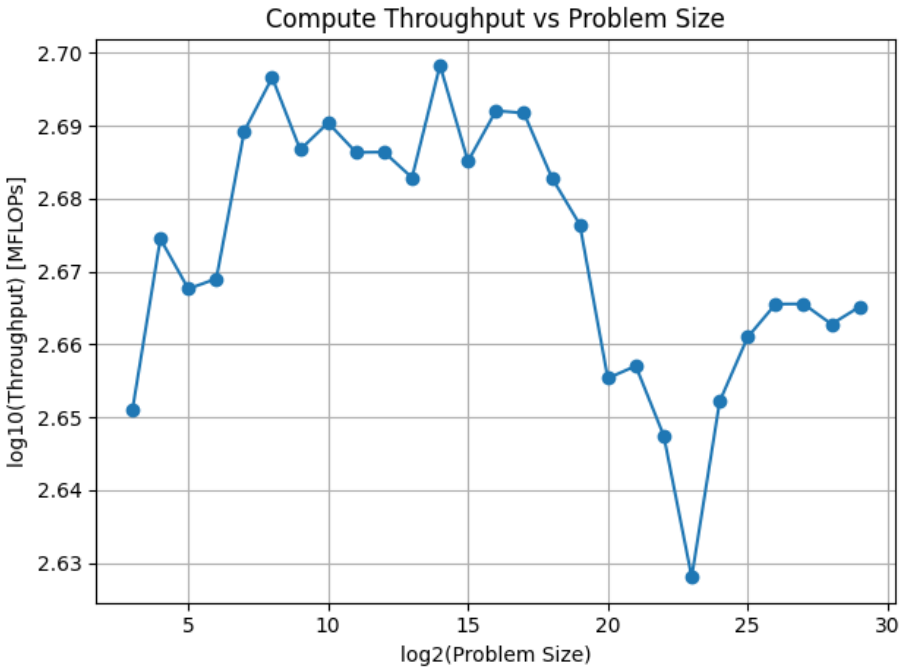


Figure 15: Throughput vs. Problem Size for the vector triad operation using HPC Cluster.

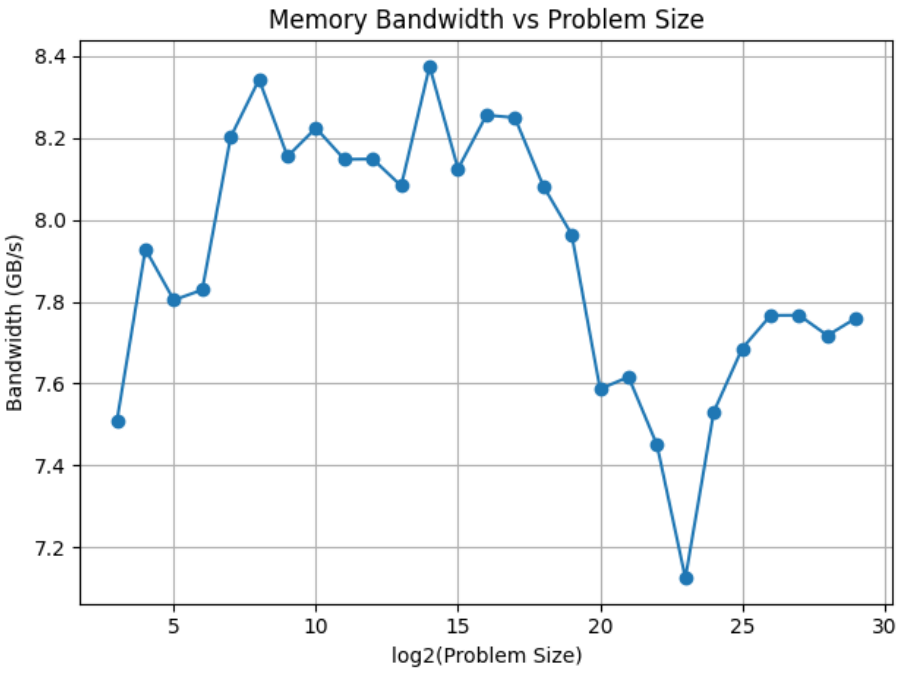


Figure 16: FLOPs vs. Problem Size for the vector triad operation using HPC Cluster.

## Analysis Tasks

### 1. Runtime Measurement and Timing Functions

Accurate runtime measurement is essential for reliable performance analysis. In this experiment, execution time is measured using high-resolution timing functions.

- The function `clock_gettime()` with the `CLOCK_MONOTONIC` clock is used.
- It provides high-resolution wall-clock time.
- It is not affected by system clock changes.
- It accurately captures delays caused by memory access.

The `clock()` function measures CPU time rather than wall-clock time and has lower resolution. Hence, it does not accurately reflect memory access delays and is not suitable for this experiment.

### 2. Bandwidth vs Working Data Size

Memory bandwidth is calculated as:

$$\text{Bandwidth} = \frac{\text{Total data transferred (Bytes)}}{\text{Execution time (seconds)}}$$

The observed trends are:

- High bandwidth for small working data sizes.
- Gradual reduction in bandwidth as data size increases.
- Performance drop occurs when data no longer fits in cache.

This behavior occurs because memory access transitions from cache memory to slower main memory.

### 3. Performance vs Working Data Size

Performance is measured in terms of throughput or FLOPs per second.

- For small problem sizes, performance remains almost constant.
- As working data size increases, performance degrades.
- Memory access latency dominates execution time for large data sizes.

This indicates that the benchmark is primarily memory-bound.

## 4. Computation Time vs Memory Access Time in the Triad Kernel

The triad kernel performs the operation:

$$a[i] = b[i] + c[i] \times d[i]$$

Each iteration involves:

- Three memory reads: $b[i], c[i], d[i]$
- One memory write: $a[i]$
- One multiplication and one addition

Although the arithmetic operations are fast, memory access dominates execution time. Therefore, the triad kernel is clearly memory-bound rather than compute-bound.

## 5. Measured vs Theoretical Peak Performance

Theoretical peak performance represents the maximum achievable performance under ideal conditions.

However, measured performance is lower due to:

- Limited memory bandwidth
- Cache misses
- Memory latency
- System and OS overheads

Since the benchmark stresses memory access more than computation, memory bandwidth becomes the limiting factor.

## 6. Impact of Cache Sizes on Performance

Cache memory plays a crucial role in determining performance.

- When working data fits in cache, performance is high.
- As data size exceeds cache capacity, cache misses increase.
- Sudden drops in performance graphs indicate cache boundary crossings.
- Further degradation occurs when accessing main memory.

These observations highlight the importance of cache hierarchy in high-performance computing systems.

## 11 Conclusion

In this experiment, the effect of CPU architecture and memory hierarchy on program performance was studied using STREAM-like benchmark kernels. The results show that cache memory plays a major role in determining performance. For small problem sizes, high performance is achieved due to fast cache access. As the working data size increases, memory access becomes the main bottleneck, leading to reduced throughput and bandwidth. This experiment clearly demonstrates the importance of memory hierarchy in high-performance computing.